

Open in app ↗

Get unlimited access to the best of Medium for less than \$1/week. [Become a member](#)

Advanced Graphics Programming in Unreal, part 1

8 min read · Jun 24, 2025



William Mishra-Manning



Listen



Share

... More

This is a seven-part article series.

Part 1: Introduction

Part 2: [Rendering Abstractions \(RHI, RDG\) and Threading](#)

Part 3: [Scenes, Views, and SceneViewExtension](#)

Part 4: [Global Shaders](#)

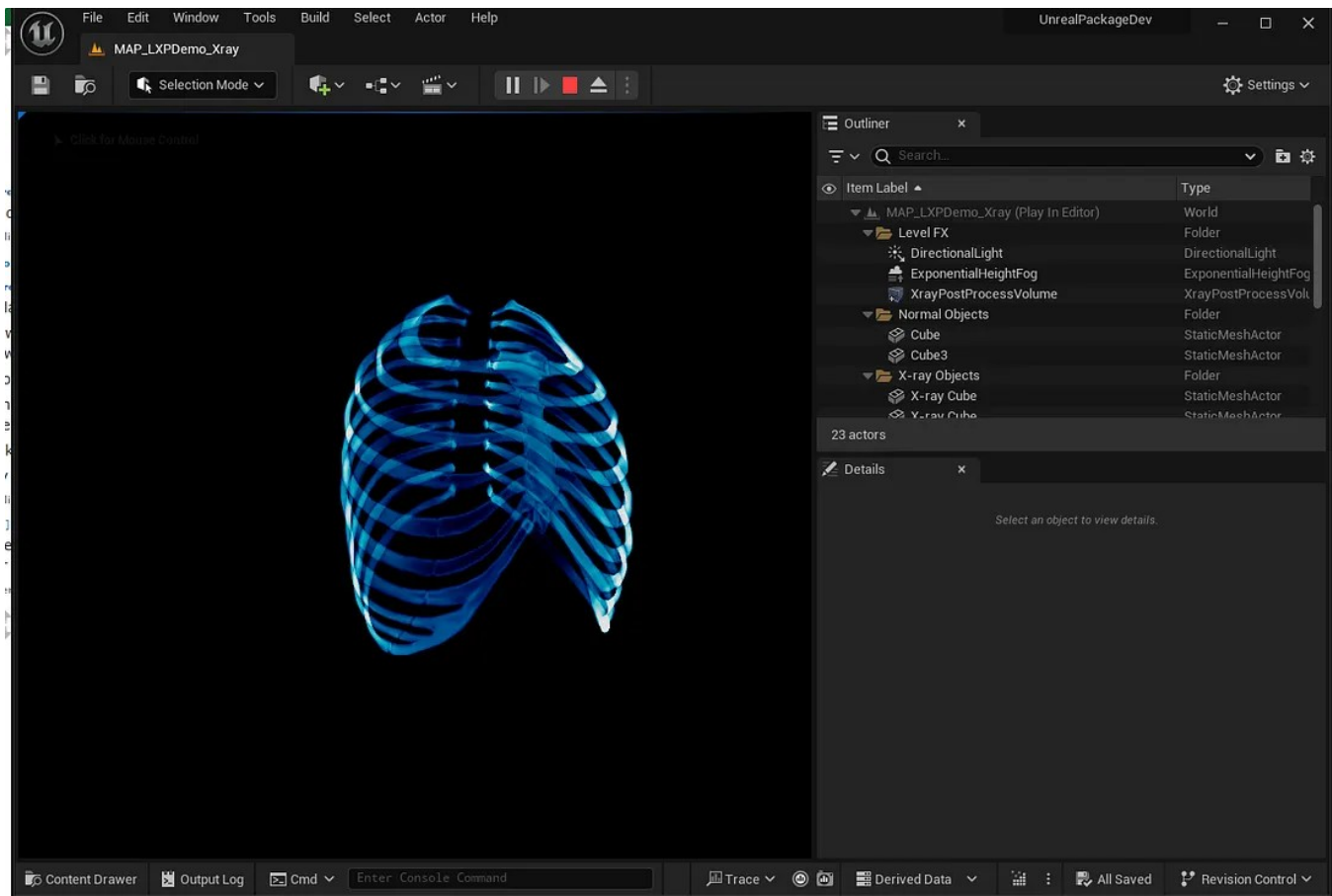
Part 5: [Simple Material Shaders](#)

Part 6: [Advanced Material Shaders](#)

Part 7: [Mesh-Material Shaders and Mesh Passes](#)

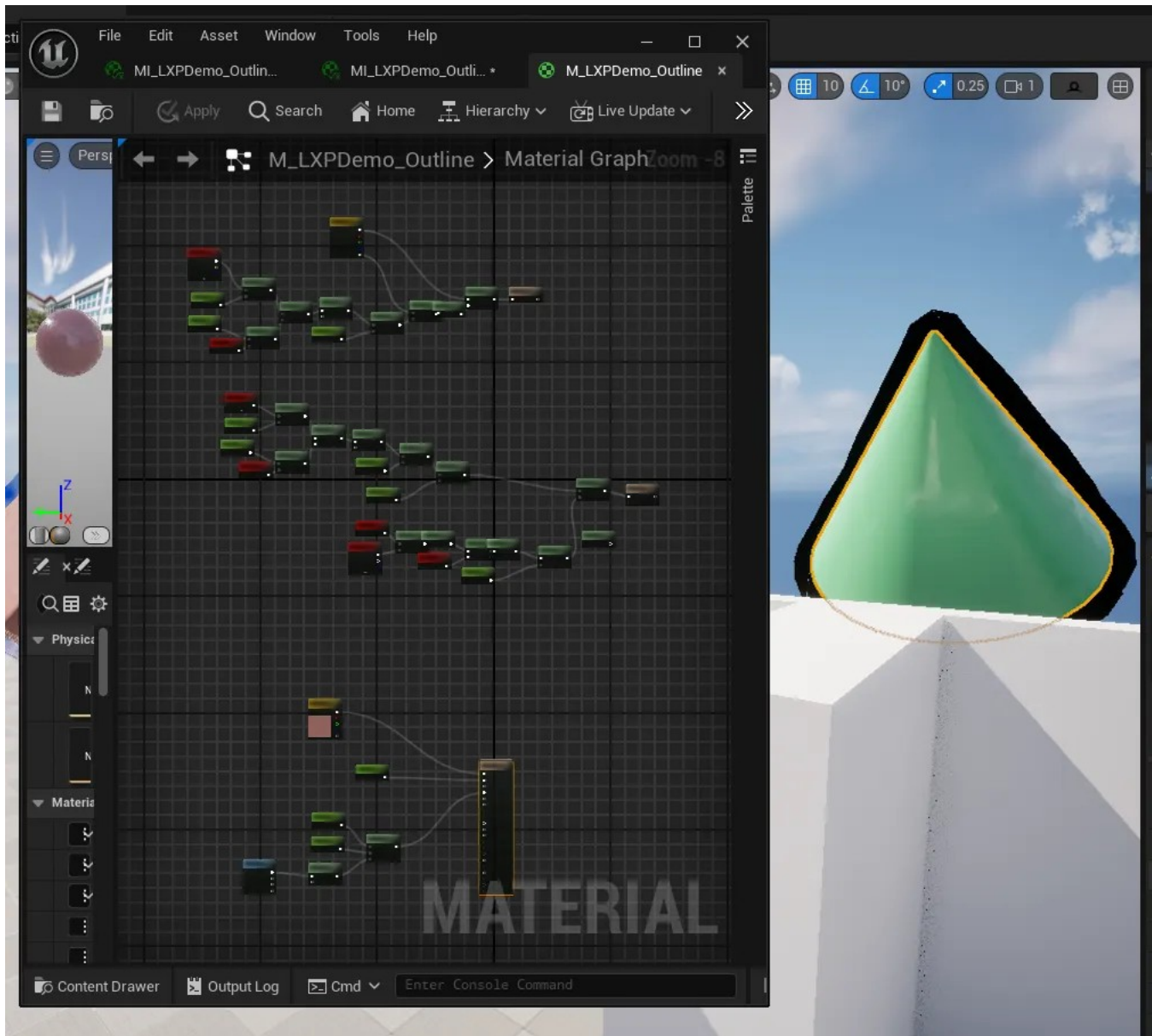
Empower Tech Artists with custom Material graphs and mesh passes

Want to make this in Unreal?

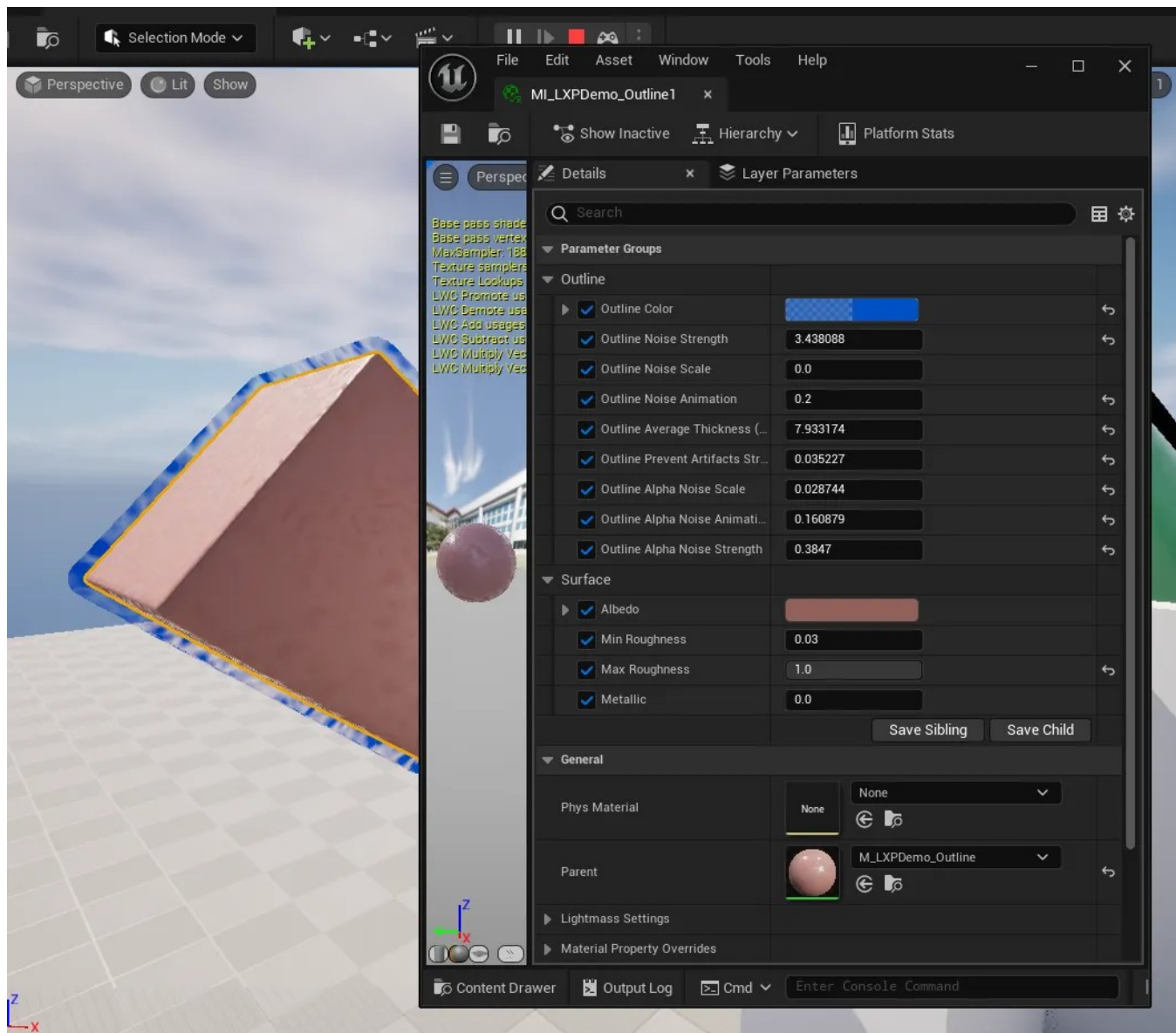


A decently-realistic yet real-time Xray renderer, made for work

Or this?



An object outline effect, controlled by the object's own Material using extra outputs



Another object in the same scene, rendering a very differently outline controlled by MaterialInstance

How about this?

Another Unreal effect, from my personal project Leave Them Behind

These are all possible in Unreal 5.3, without modifying the engine. However you can't easily do any of them with only Materials and Blueprints; most are complex multi-pass effects, with 3D mesh passes *and* custom screen passes *and* off-screen compute shaders. The Outline effect is easier, perhaps doable though BP and existing Materials, but is still clumsy to implement that way.



The Hiss, from the game Control, use a very complex effect where particles draw into a screen-space fluid simulation which is reprojected every frame as the camera moves. Can this be done in Unreal without modifying the engine?

Additionally, to enable fast iteration on the effects by artists, these custom passes are all driven by Materials with new kinds of input and output pins. There are no hard-coded parameters for programmers to feed into the shader by hand!

Don't underestimate the power that Material integration gives a graphics programmer. With it you no longer have to make your own judgement calls on how to parameterize an effect for your artists, through HLSL then C++ then Blueprints/Reflection. Instead you can leverage the massive power and user-friendliness of Material graphs to let artists take ownership over those parameters and the logic that relates them. It provides an order-of-magnitude speedup to iterating on complex render effects.

Unfortunately all this architectural power is hidden under a morass of under-documented C++ code, *undocumented* C++ code, countless interacting structs and classes, tricky threading concerns, and bewildering type names. Nothing I've seen on the internet gets further than making a screen-space Global Shader and maybe a

Scene-View Extension to invoke it. I suspect the only people that know how to go further are AAA graphics programmers, Epic engineers, and now partially myself. For work and my own personal projects I've poured over Unreal rendering samples, plus the few docs they do have, to figure out custom mesh passes and Material integration. What I found is a hugely-customizable render pipeline, but only if you have a small novel's worth of notes to reference as you work.

Outline

The goal of this series of articles is to provide three awesome things:

- A re-usable Unreal plugin that greatly simplifies screen-space/compute passes, 3D mesh passes, and Material integration.
- A reference for all the information I've uncovered so far, and any questions I still have.
- A demo of these features in action, which we build step by step.

We are working in unreal 5.3, 5.4, or 5.5, and I assume you have the following knowledge already:

- **C++.** Most of this code doesn't interact with game logic or UObjects; it's pure modern C++ within an enormous C++ environment. In particular you should understand move-semantics, lambdas, and lambda capturing, because they're all vital tools for the next topic...
- **Multithreading.** Unreal uses numerous threads within its renderer, and you should be comfortable with multithreaded thinking. I *will* explain Unreal-specific concepts, conventions and design patterns. There is also a [good official page on this topic](#).
- **Graphics Programming.** Nothing Unreal specific! Just HLSL, the anatomy of a draw call, and the general nature/lifetime of graphics objects.
- **Basic Unreal skills.** I won't waste time explaining the blueprints involved in making the demo, however the released project will contain it all if you don't want to build it from scratch.

Each article will be split into four sections:

1. **Theory.** A straightforward explanation of the topic.
2. **Demo.** Applying the topic to a custom effect we build throughout the series.
3. **Code.** A GitHub repo containing the demo code.
4. **Details.** All the precise implementation details I've found and any outstanding questions I have.

Demo

This is the final effect we are aiming for:

The effect running on static meshes

The effect running on Niagara particles

The effect running on skeletal meshes

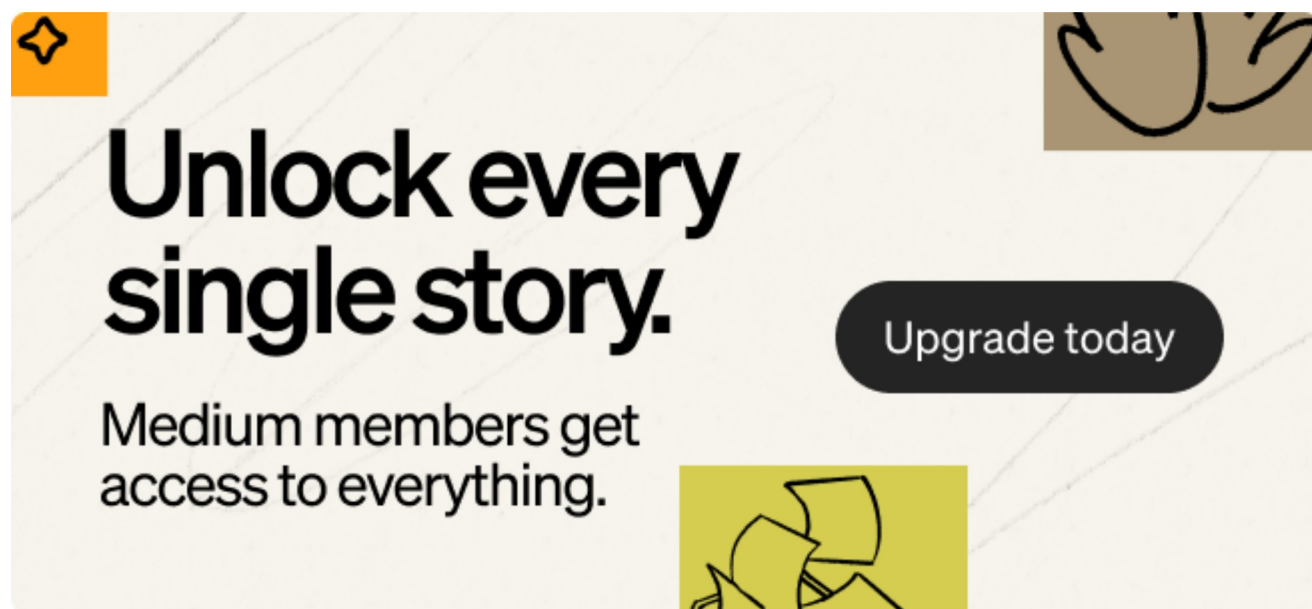
As you may notice, it's a screen-space Game of Life — a very popular Cellular

Automata simulation you can read about elsewhere — which is affected by meshes in the scene. I chose the Game of Life because it's very simple to implement.



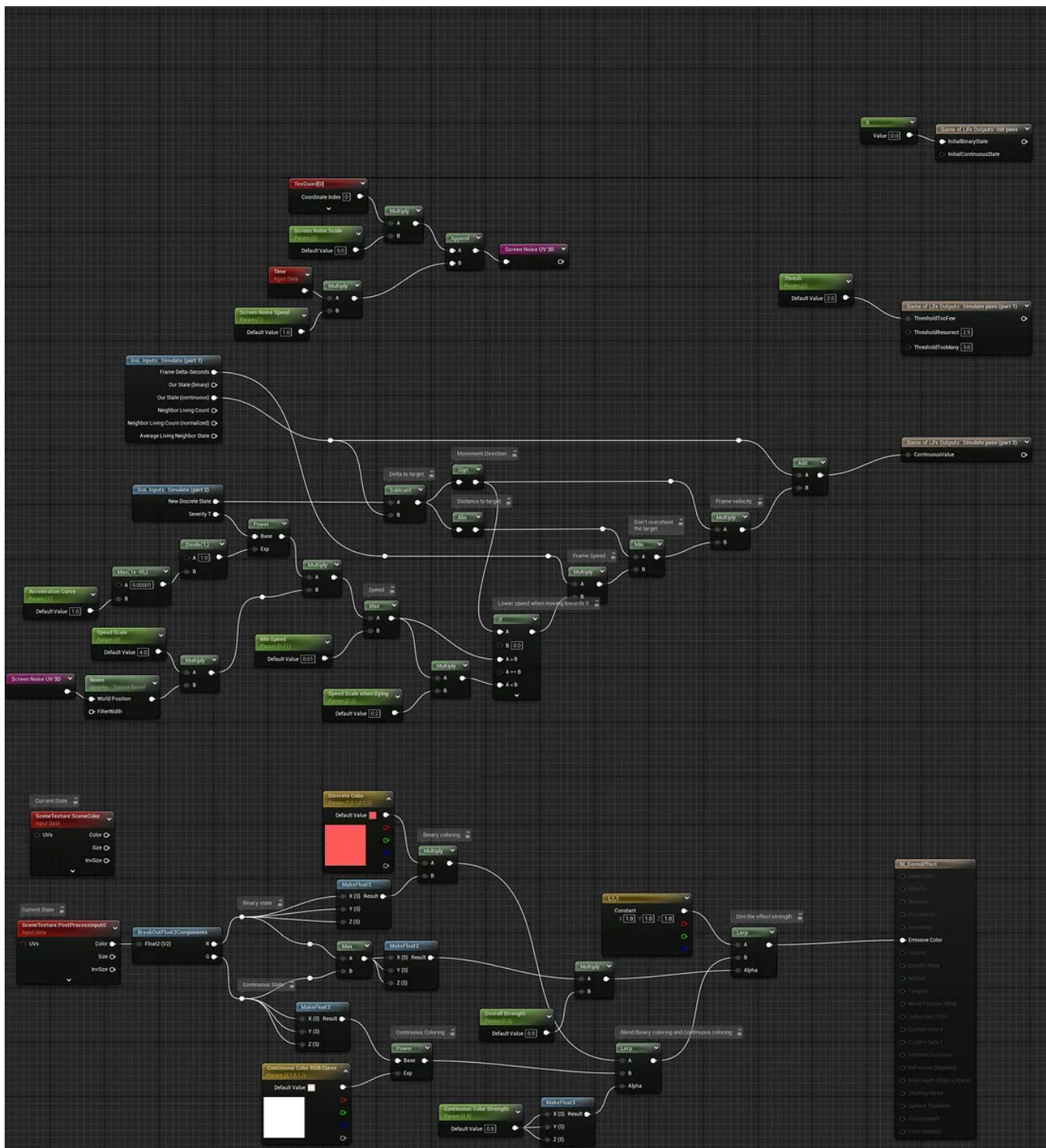
A plain example of the Game of Life, courtesy of Wikipedia

This effect requires four different passes: *Initialize* to set the initial state, a 3D *Mesh* pass to let meshes/primitives draw into the sim, a *Simulate* compute shader to update the sim, and a *Display* pixel shader to apply the sim to the screen. In the end, all 4 of them will be controlled by Materials. There will also be a small *Resample* shader to run whenever the player's resolution changes (which actually happens very often in the editor, or when playing with quality settings).



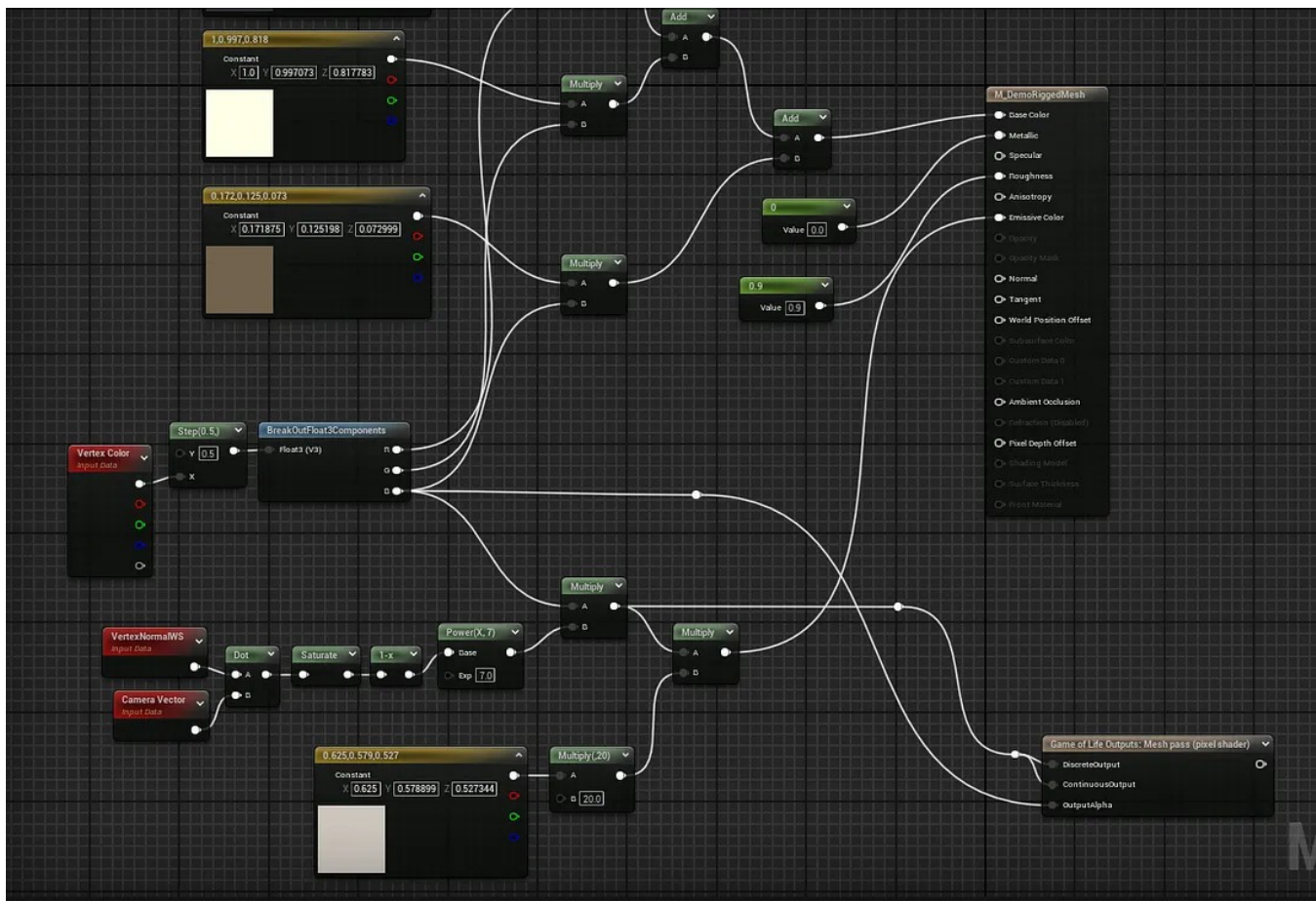
The mesh pass will be controlled by each object's own Material, while the screen passes are all controlled by one global Material. To give you an idea of where we are going, here is a preview of the final Material graph which drives the screen-space shaders for this effect. It has three different sub-graphs, one for each shader

(Initialize, Simulate, Display), and offers many parameters which make it easy to iterate on it or to shift the behavior dynamically at runtime.

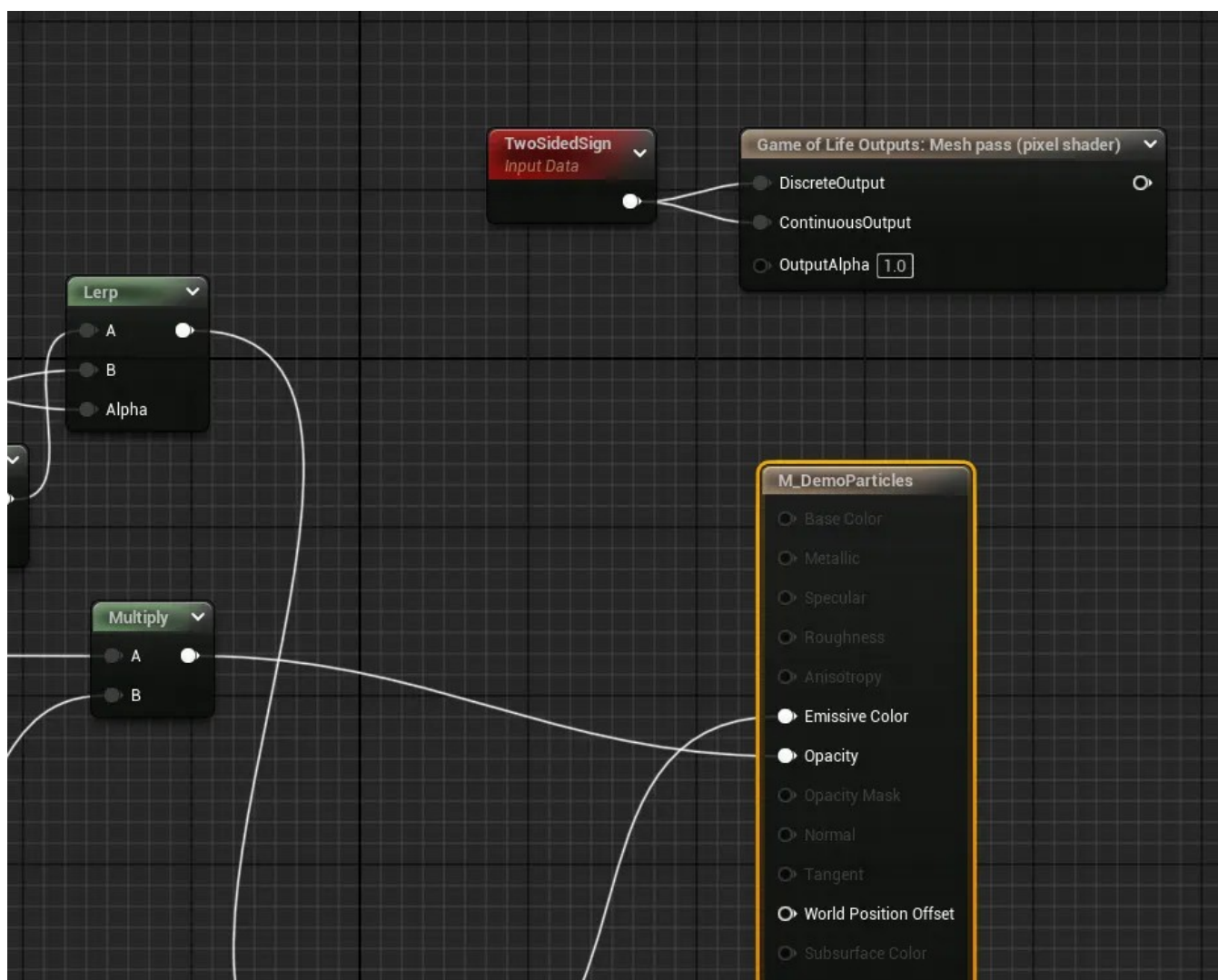


The Game of Life effect's main 3 shaders are driven by this material.

Here are some of the materials of objects participating in the mesh pass:



The rigged mesh has three segments, masked by vertex color. The head is masked with Blue, and drawn with a fresnel effect. The head also draws into the sim using that fresnel effect.



The niagara particles draw into the sim with multiply blending (configured elsewhere), and either kill or revive cells based on which side of them is visible.

I will name this demo project “GoL” for “Game of Life”. So, let’s get a project set up for graphics programming!

Setup

Start with an empty Unreal C++ project. Drop in [my reusable ExtendedGraphicsProgramming plugin](#). Create your own new plugin (in the editor go to *Edit>Plugins*, click the +Add button, and choose Blank Plugin) to hold the demo’s shaders and graphics code.

This plugin will need at least one module, of Type *Runtime* and LoadingPhase *PostConfigInit* (shaders must load in the *PostConfigInit* phase). The module will have

some extra private dependencies on the engine modules *Projects*, *RenderCore*, *Renderer*, *RHICore*, and *RHI*.

The plugin should also contain at least one folder to hold your shaders, conventionally named *Shaders* and placed alongside *Source*. More complex plugins may want to separate them into *Shaders/Public* and *Shaders/Private*.

Finally we need to tell Unreal about our *Shaders* folder, by assigning it a virtual path that can be referenced in other shaders (and by Material graphs, in their Custom nodes). In your module's startup code, call

```
AddShaderSourceDirectoryMapping(virtualFolderPath, actualFolderPath)
```

For example:

```
AddShaderSourceDirectoryMapping("/MyShaders", "C:/Users/manni/Documents/MyGame/
```

Obviously it's not a great idea to use hard-coded paths, so instead we will ask Unreal for the local path to the plugin:

```
#include "Interfaces/IPluginManager.h"
...
auto myPlugin = IPluginManager::Get().FindPlugin(
    TEXT("MyPlugin") //Your plugin's name here
);
auto myShadersDir = FPaths::Combine(
    myPlugin->GetBaseDir(),
    TEXT("Shaders") //Relative to the plugin's root folder
);
AddShaderSourceDirectoryMapping("/MyShaders", myShadersDir);
```

Now Unreal will know about your folder when compiling shaders! You can add sub-folders, headers (.ush files), and actual shaders (.usf files) to that folder and reference them in other places using your virtual path. For example:

```
#include "/MyShaders/myHeader.ush"
```

Debugging

I highly recommend building the engine locally and targeting Debug Editor (not just DebugGame Editor). As you experiment you will probably cause crashes deep within the engine, and it really helps to be able to step through every layer of abstraction.

You should also prepare Unreal for graphics debugging by going into the engine's *Config/ConsoleVariables.ini* file and enabling most of the graphics-related debug flags that you can find commented out there:

- *r.ShaderDevelopmentMode=1*
- *r.ShaderCompiler.EmitWarningsOnLoad=1* : Unfortunately, this will spam *tons* of warnings for built-in shaders, so maybe just skip it.
- *r.RHICmdBypass=1* : This turns off low-level threading in the renderer for easier step-through debugging and tracing of crashes. To completely turn off threaded rendering, enable the next command as well. However it must be turned off at some point to test that your code is thread-safe! The next article will explain where exactly “RHI” fits in the rendering architecture.
- *r.RDG.ImmediateMode=1* : This turns off high-level threading in the renderer for easier step-through debugging and tracing of crashes. To completely turn off threaded rendering, enable the previous command as well. However it must be turned off at some point to test that your code is thread-safe! The next article will explain where exactly “RDG” fits in the rendering architecture.
- *r.RDG.Debug=1* : This enables valuable render pipeline checks that would

normally be skipped for performance.

I also highly recommend getting RenderDoc and enabling its Unreal plugin for the project (already built into the engine). This lets you do awesome low-level debugging, like stepping through shaders. If you use this, then you should enable the following commands to make debugging in RenderDoc far easier:

- *r.Shaders.Optimize=0*
- *r.Shaders.Symbols=1*
- *r.Shaders.ExtraData=1*
- *r.Shaders.SkipCompression=1*

Finally, know how to switch between graphics API's (which Unreal calls "RHI"s) in the project settings. In my experience each API reacts totally differently to the bugs I introduce, so I have to switch between them frequently to find one that allows me to start debugging. DX12 likes to throw a generic "failed to create GraphicsPipeline" error (in my experience it usually means vertex outputs didn't match fragment inputs), or hang forever on a particular framebuffer function. Vulkan can crash in an opaque and confusing location deep in its RHI thread. DX11 tries really hard to paper over the problems in your code.

To change the default RHI, go into the project settings and just search for "RHI". By default it's set to "Default" but you can pick a specific one and then restart the editor. To see the current RHI in use, execute the console command `r.RHI.Name` and read the output in the console.

Test Scene

Create a scene with several different kinds of materials and primitives. My scene has the following variety of objects that will all participate in the effect:

1. Static meshes with an animated material
2. Static meshes with multiple materials, and multiple LOD's. The LOD's are deliberately aggressive and ugly, to make it obvious which level is visible.

3. Niagara particles

4. Rigged “skeletal” meshes, animated procedurally with this snippet:

```
boneRot = rotMax * perlin(elapsedTime*C + uniqueId*D)
```

Each of these will participate in the effect in its own way, demonstrating that our code works with virtually any type of Primitive Component.

Code

Part 1 of the demo can be found in the below repo, in the branch *part/1*. It holds my “Extended Graphics Programming” plugin as a git submodule.

GitHub - heyx3/UnrealExtendedGraphicsProgrammingDemo: Demo code for the Extended Graphics...

Demo code for the Extended Graphics Programming unreal plugin
(also on github) ...

github.com

Details

No extra info for this first article!

Unreal Engine 5

Graphics Programming

Shaders

Technical Art



Edit profile

Written by William Mishra-Manning

59 followers · 5 following

Graphics Programming, Procedural Generation

Responses (1)



William Mishra-Manning

What are your thoughts?



Matthew Vecchio

Sep 2, 2025



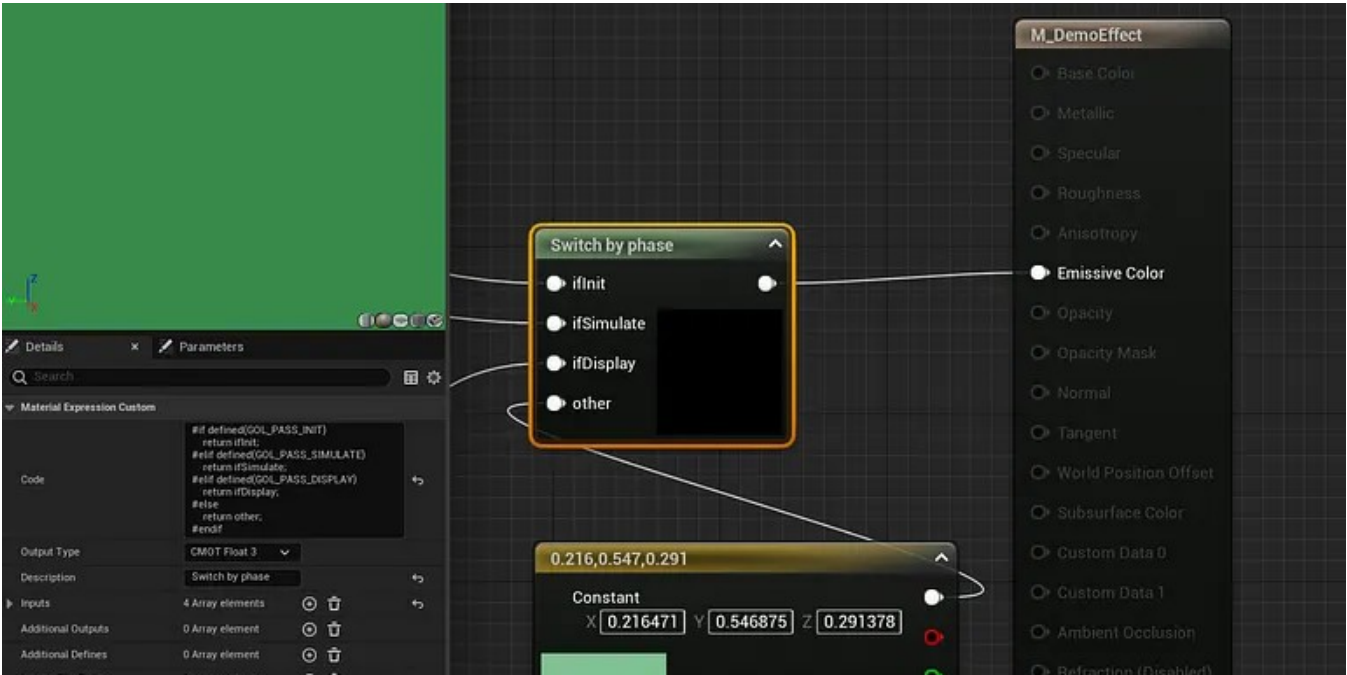
Thank you this is an invaluable resource.



1

[Reply](#)

More from William Mishra-Manning



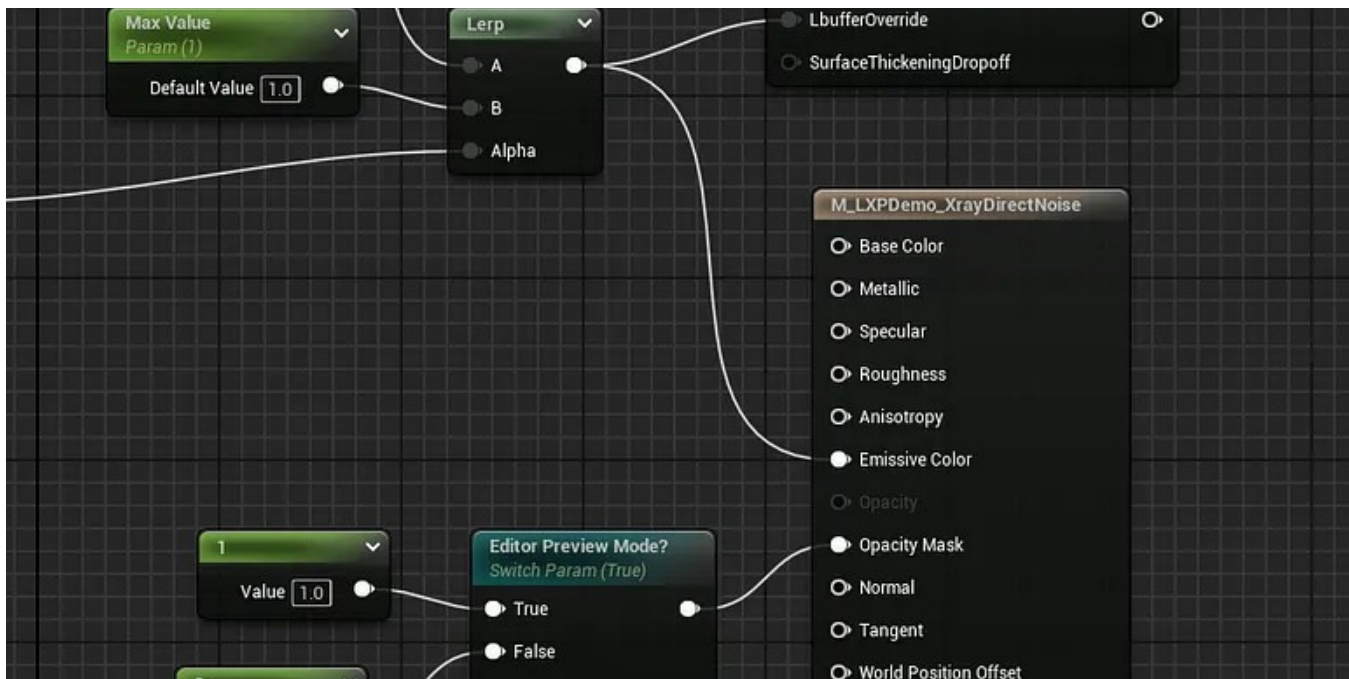
 William Mishra-Manning

Advanced Graphics Programming in Unreal, part 5

This is a seven-part article series.

Jun 25, 2025  10



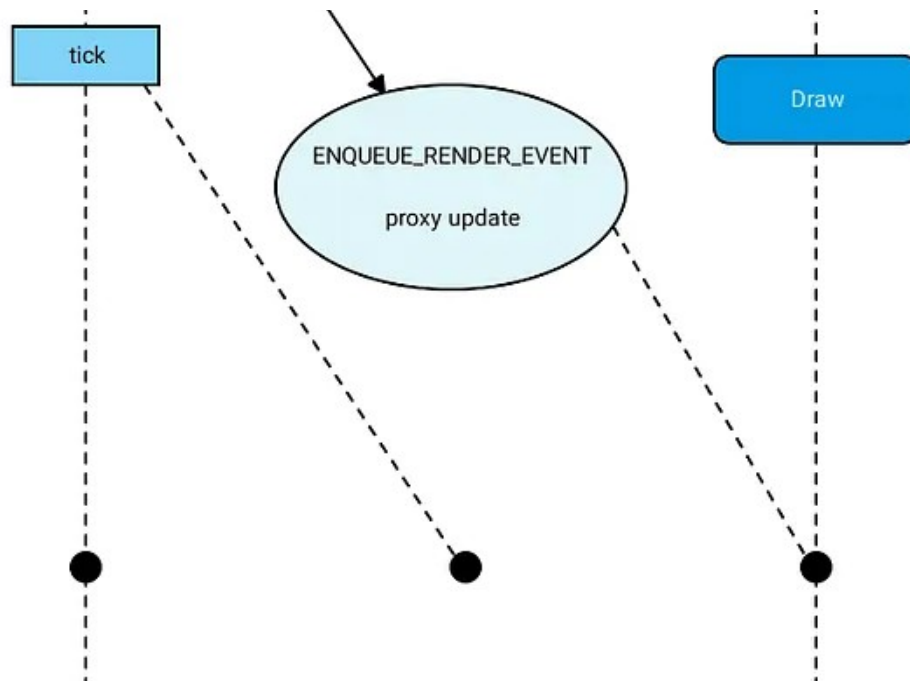


 William Mishra-Manning

Advanced Graphics Programming in Unreal, part 6

This is a seven-part article series.

Jun 25, 2025  1  2





William Mishra-Manning

Advanced Graphics Programming in Unreal, part 2

This is a seven-part article series.



William Mishra-Manning

Advanced Graphics Programming in Unreal, part 3

This is a seven-part article series.

Jun 24, 2025



6



See all from William Mishra-Manning

Recommended from Medium



James Roha

Gaussian splatting: a complete student guide to 3D capture in 2026

3D Gaussian Splatting has transformed how we capture and render real-world scenes, enabling photorealistic results at 100+ frames per...

Feb 13



7

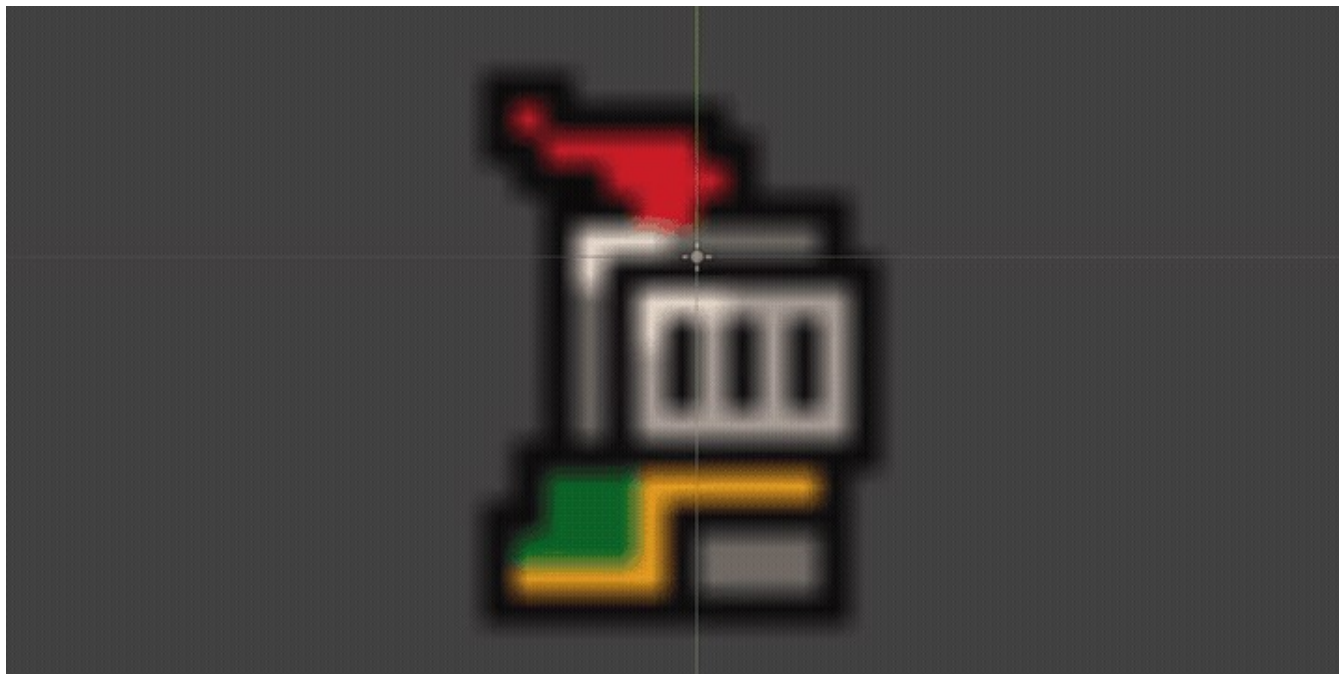


 AmirHossein Aghajari

Liquid Glass: iOS Effect Explanation

Apple's Liquid Glass effect with shaders: math, distortion & chromatic aberration

Nov 23, 2025  240  1



 In CodeX by Mina Pêcheux 

Doing pixel-perfect in Godot the right way!

The one trick that changed my pixel-art life

★ Nov 27, 2025  15





a Akash Kadam

Setting Up FreeRTOS on STM32 / ESP32 from Zero A Hands-On Beginner's Guide

If you've already understood what FreeRTOS is and why bare-metal eventually breaks, the next natural question is:

Jan 29 🖱 1





Nikhil Maurya

What Unreal doesn't tell you about shader permutations

Shader permutations are one of those Unreal Engine systems you don't really think about. On the surface, materials feel flexible, modular...

Jan 1 🖱 5



1. Start with Bayes' Rule:

$$P(x|z) = \frac{P(z|x)P(x)}{P(z)}$$

2. Divide by the "Free" state ($\neg x$) to create an Odds Ratio:

$$\frac{P(x|z)}{P(\neg x|z)} = \frac{P(z|x)}{P(z|\neg x)} \cdot \frac{P(x)}{P(\neg x)}$$

3. The Result: This allows the robot to update its map by comparing how likely a measurement is if a wall exists ($P(z|x)$) versus if it doesn't ($P(z|\neg x)$).



Naren Suri

Occupancy Grid Mapping Algorithm

Occupancy Grid Mapping serves as a foundational framework in robotic perception, enabling a mobile agent to represent an unstructured...

Mar 20 🖱 1

[See more recommendations](#)